

SVR Indian Oil Service Station

Software Installation & Troubleshooting Guide — New PC Setup

1. Software to Download & Install on a New PC

End-User Station PC (the actual pump-side computer)

Only one download is needed: the single packaged Windows installer (.exe). This bundles the application itself — you do not separately download or install Python, Node.js, or SQLite on the station PC.

- Windows 10 or 11 (64-bit) — the PC's operating system, not something you install separately
- The SVR application installer (.exe) — this single file contains everything the station PC needs

What's Inside the Installer (bundled automatically, no separate download)

- ElectronJS — the desktop application shell (what you actually see and click on)
- Python — runs the business logic and calculations in the background
- SQLite — the database that stores all your daily entries

Tesseract OCR — used for the Scan/Upload feature. CONFIRMED as part of the software requirements, but NOT explicitly confirmed as bundled inside the single installer the same way Python/SQLite/ElectronJS are. This is a real gap worth resolving before build, not an assumption made here — see the note below.

OPEN ITEM: Tesseract OCR's installation method is not yet finalized. It may need to be bundled into the installer separately, or installed as its own step. Confirm this with the development team before the first real installer build — it affects whether Scan/Upload works immediately after installation or needs an extra setup step.

Development / Build Machine Only (not needed on the station PC)

These are only for whoever builds the installer itself — never needed on the actual pump station's PC:

- Python 3.x (exact version to be confirmed by the development team)
- Node.js + npm (used to build the ElectronJS frontend)
- Git (source control — project repository at github.com/Mithendra/IOCL_SVR)
- An Electron packaging tool such as electron-builder (used to produce the final .exe)

2. Do You Need All of This? Is a Single .exe Enough?

Short answer: yes, one installer is enough for the station PC. You do not need to separately install ElectronJS, Python, or SQLite — the single .exe installer bundles all three together. This is already a confirmed design decision, not a proposal.

The list of technologies (ElectronJS, Python, SQLite) describes what the application is built with, not what you personally need to download and manage. Think of it like any other Windows program — you install one file, and everything that program needs comes along with it.

The one caveat is Tesseract OCR (see the Open Item above) — its bundling method isn't finalized yet. Everything else is confirmed as a single-installer experience.

3. Installation Instructions

Step 1 — Obtain the Installer

Get the installer .exe file — either from the project's build release, or a copy provided directly to you.

Step 2 — Run as Administrator

Right-click the installer file and choose "Run as administrator". This step is required — the installer needs to register Windows Services (background components that keep the system running), which needs elevated permission.

Step 3 — Let the Installer Finish

The installer will: install the application files, create the database file (on first run), set up the log folder structure, and register each component (backend, frontend, scheduler) as a Windows Service set to start automatically.

Step 4 — Confirm Everything Started

Once the installer finishes, every component should already be running — no manual launching needed. On every future restart of the PC, the same happens automatically via Windows Services.

Verifying Services Manually

- Press Win + R, type services.msc, press Enter
- Look for the SVR-IOCL service entries (proposed names: "SVR-IOCL Backend Service", "SVR-IOCL Frontend Service", "SVR-IOCL Scheduler Service")
- Right-click a service > Properties to see its Status (should say Running) and Startup Type (should say Automatic)

This tells you WHETHER a component is running, but not WHY it failed if it isn't — for that, see the Troubleshooting section below.

4. Troubleshooting: When Startup Fails

Every component writes to its OWN separate log file — not one shared/mixed log. This means you can go straight to the log file for whichever component failed, rather than searching through everything.

Component	Log File Location
Python backend service	C:\ProgramData\SVR-IOCL\logs\backend-service.log
SQLite / database layer	C:\ProgramData\SVR-IOCL\logs\database.log
ElectronJS frontend	C:\ProgramData\SVR-IOCL\logs\frontend.log
Scheduled job (23:59 IST carry-forward)	C:\ProgramData\SVR-IOCL\logs\scheduler.log
Email integration (password reset, reports)	C:\ProgramData\SVR-IOCL\logs\email-integration.log
OCR / Tesseract processing (Scan/Upload)	C:\ProgramData\SVR-IOCL\logs\ocr-processing.log

These live in C:\ProgramData (a system-wide folder) rather than a personal user folder, because the services run at the system level — not tied to whichever staff member happens to be logged in that shift.

Step-by-Step: A Service Won't Start

- Open services.msc and confirm which specific service shows Stopped
- Open that component's dedicated log file from the table above (not the others — go straight to the right one)
- Look at the most recent entries at the bottom of the file for an error message and timestamp
- Common causes: the database file is locked or missing, a required port is already in use, or a Windows Update changed a permission — the log's error text will point to which
- If the log file itself doesn't exist yet, the service may have never started even once — check Windows Event Viewer (search "Event Viewer" in the Start menu) for installer-level errors

OPEN ITEM: exact Windows Service names, the database file's own path, and whether the log retention period matches the confirmed 1-year policy for application data or uses a shorter rotation, are not yet finalized — confirm with the development team before relying on these details in a live deployment.

This guide reflects confirmed decisions in the SVR BRD (Section 4.6) as of 2026-08-26. Items marked OPEN ITEM are proposed, following standard Windows conventions, but not yet finalized in the design phase.